

Universidad Rafael Landívar
Facultad de Ingeniería
Curso: Compiladores

Catedrático: Mgtr. Diana Gutiérrez

MANUAL DE USUARIO: MINILANG (FASE 3)

Ana Paula Ortiz Hernandez - 1186624
Juan Pablo Mazariegos Sepúlveda - 1140024

Guatemala 18 de mayo de 2026

1. Introducción

El presente documento describe el funcionamiento del compilador desarrollado para la Fase #3 del curso de Compiladores. El sistema, denominado **MiniLang**, implementa de forma incremental el análisis léxico, sintáctico y, de manera central en esta fase, el **análisis semántico** con gestión dinámica de una tabla de símbolos y comprobación estricta de tipos con soporte para coerción.

2. Objetivos

- **Objetivo General:** Implementar la fase de análisis semántico y validación de tipos para el lenguaje MiniLang, integrándolo con los analizadores previos para garantizar la construcción correcta de programas válidos semánticamente.
- **Objetivos Específicos:**
 - Construir y manejar dinámicamente una estructura eficiente para la Tabla de Símbolos, exportando su estado final a un archivo de salida.
 - Validar la compatibilidad de tipos en expresiones, asignaciones, operaciones aritméticas y llamadas a funciones.
 - Implementar un sistema de recuperación de errores semánticos que permita al compilador continuar el análisis hasta el fin del archivo (EOF).

3. Requisitos del Sistema

Software requerido

- **Lenguaje de programación:** C# (.NET 8.0 / .NET 9.0)
- **IDE utilizado:** Visual Studio 2022 / VS Code
- **Librerías utilizadas:** Ninguna externa (Análisis nativo mediante clases estructuradas).

Hardware recomendado

- **Procesador:** Intel Core i3 / AMD Ryzen 3 o superior.
- **Memoria RAM:** 4 GB mínimo.
- **Espacio disponible:** 100 MB de espacio libre en disco.

4. Instalación y Configuración

- Paso 1: Clonar o descargar el proyecto desde el repositorio en GitHub.
 - Enlace del repositorio: [https://github.com/\[TuUsuario\]/\[TuRepositorio\]](https://github.com/[TuUsuario]/[TuRepositorio]) (Recuerda agregar a la usuaria *diana1190*).
- Paso 2: Abrir el archivo de solución `.sln` del proyecto en Visual Studio.
- Paso 3: Restaurar las dependencias de .NET y compilar la solución desde el IDE o mediante la terminal.

5. Ejecución del Compilador

El ejecutable compilado se llama `minilang`. Para ejecutarlo desde la consola de comandos, navega a la carpeta de salida y ejecuta:

```
Bash
minilang.exe ruta/del/archivo.ming
```

Nota: El compilador solicita obligatoriamente la ruta de un archivo con extensión `.ming` como argumento de entrada.

6. Funcionamiento del Sistema

6.1 Análisis Léxico

El scanner lee el flujo de caracteres del archivo `.ming` y reconoce tokens válidos, palabras reservadas (`class`, `Program`, `int`, `double`, `string`, `void`, `if`, `else`, `while`, `const`, `input`), operadores y delimitadores. Reporta cualquier símbolo inválido con su respectiva línea y columna.

6.2 Análisis Sintáctico

El parser valida que la secuencia de tokens se ajuste estrictamente a la gramática formal definida para MiniLang. Construye el árbol de derivación o ejecuta el análisis garantizando el orden correcto de declaraciones, estructuras y bloques jerárquicos.

6.3 Análisis Semántico & Tabla de Símbolos

Esta fase ejecuta las siguientes comprobaciones clave:

- **Tabla de Símbolos:** Almacena y procesa identificadores (variables, constantes, funciones) con sus tipos y ámbitos (scopes) de manera dinámica. Genera un archivo de salida independiente (`tabla_simbolos.txt`) que refleja la estructura del programa analizado.
- **Asignación y Expresiones:** Resuelve operaciones en tiempo de compilación para constantes y asigna valores directamente.
- **Coerción de Tipos:** Permite la promoción automática de tipos compatibles (por ejemplo, asignar u operar un `int` dentro de un `double`).
- **Verificación de Funciones:** Comprueba que el número y tipo de los argumentos enviados coincidan con la declaración original de la función.

- **Recuperación de Errores:** Ante un fallo semántico, guarda el error en un listado, se recupera de la instrucción fallida y continúa analizando hasta el final del archivo.

7. Características del Lenguaje (Ejemplos)

Declaración de variables y constantes

```
C#  
int m1;  
double circun;  
const double pi;  
pi = 3.1416; // Constante con valor asignado
```

Entrada y salida de datos

```
C#  
// Captura de datos y almacenamiento  
input(m1);
```

Estructuras condicionales (if-else)

```
C#  
if (a > b) {  
    c = 2;  
} else {  
    c = 3;  
}
```

Ciclos (while)

```
C#  
while (m1 > 0) {  
    m1 = m1 - 1;  
}
```

Funciones

```
C#  
int f1(int a, int b, int c) {  
    if (a > b) c = 2; else c = 3;  
    return c;  
}
```

8. Casos de Prueba Obligatorios

#	Prueba Contexto /	Código Fuente Esperado (.ming)	Resultado en Pantalla / Archivo de Salida
1	Hola Mundo	// Código MiniLang para imprimir	OK
2	Operación Aritmética	int A; double B; A = 9; B = 34.35 * A;	OK (Aplica coerción de int a double)
3	Uso de input e if	int x; input(x); if (x > 10) { ... }	OK
4	Función y Retorno	class Program { ... MyParser.fl(1,2,3); }	OK (Parámetros y tipos correctos)
5	Uso de while	while (condicion) { ... }	OK
6	Error Léxico	int @valor = 5;	line X, column Y: ERROR: Símbolo inválido '@'
7	Error Sintáctico	int fl(int a (sin cerrar paréntesis)	line X, column Y: ERROR: Token inesperado
8	Error Semántico	string A; int B; A = "hola"; B = 34 * A;	line 2, column 9: ERROR: Cannot operate type 'int' and 'string'

9. Manejo de Errores

El sistema centraliza los errores detectados en un formato uniforme para facilitar el depurado por parte del usuario:

- **Error Léxico:** Captura caracteres que no pertenecen al alfabeto de MiniLang.
 - *Ejemplo:* line 4, column 12: ERROR: Símbolo inválido '\$'
- **Error Sintáctico:** Se dispara cuando la estructura gramatical es incorrecta.
 - *Ejemplo:* line 8, column 5: ERROR: Se esperaba ';' después de la expresión
- **Error Semántico:** Valida incompatibilidad de tipos y variables no declaradas o duplicadas.
 - *Ejemplo:* line 11, column 26: ERROR An argument for function f1 is invalid

10. Estructura del Proyecto

La estructura se organiza de la siguiente manera:

Plaintext

Proyecto/

```
|
|—— src/           # Código fuente del compilador en C# (.cs) [cite: 70, 161]
|—— pruebas/       # Los 8 archivos de prueba obligatorios (.ming) [ 71, 163,
170]
|—— resultados/    # Archivos de salida de la Tabla de Símbolos generada
[72, 106, 214]
|—— documentacion/ # Este Manual de Usuario en PDF [cite: 73]
|—— README.md      # Explicación de diseño, estrategias semánticas y de
tipos 74, 195, 196]
```

11. Recomendaciones de Uso

1. **Verificación Previa:** Asegúrese de que el archivo de entrada **.ming** no contenga caracteres extraños ajenos al lenguaje especificado.
2. **Declaración Obligatoria:** Declare siempre todas las variables y constantes antes de hacer cualquier referencia u operación con ellas para evitar fallos de ámbito.
3. **Análisis por Fases:** Revise detalladamente los logs en pantalla; recuerde que un error sintáctico bloqueará un análisis semántico limpio.

12. Conclusiones

- Se integró con éxito el análisis semántico al lexer y parser previos, logrando un compilador funcional para MiniLang que aplica la teoría en una solución de software robusta y estructurada.
- Se implementó una estructura dinámica que administra correctamente el alcance (*scope*) y visibilidad de los identificadores, exportando los resultados en un archivo independiente.
- Se validaron las reglas de compatibilidad en expresiones aritméticas, lógicas y asignaciones, garantizando que cumplan estrictamente con las restricciones de MiniLang.
- Se desarrolló un sistema que reporta con claridad los errores semánticos e incluye mecanismos de recuperación para continuar el análisis del archivo sin detener el flujo.